CS & IT ENGINEERING



Algorithms

Analysis of Algorithms



Lecture No.- 05

By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Big Notations – O, Ω, Θ

Practice

Maths Revision

Small Notations.

Topics to be Covered



- Topic
- Topic
- Topic

Small Notations

Practice



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : Analysis of Algorithms

1. Big Oh (UB)

   $f(n) = O(g(n))$

   Iff $f(n) \leq c*g(n)$, some $c > 0$, $n \geq n_0$

2. Big Omega (Ω)

   $f(n) = \Omega(g(n))$

   Iff $f(n) \geq c*g(n)$, some $c > 0$, $n \geq n_0$

3. $F(n) = \Theta(g(n))$

   $f(n) = O(g(n))$

   and

   $F(n) = \Omega(g(n))$



Topic : Analysis of Algorithms

1. $f(n) = \prod_{i=1}^{n}(1) = \underbrace{1 \times 1 \times 1 \times \ldots\ldots \times 1}_{\text{n times}}$

$= 1$

$\Rightarrow O(1) \rightarrow$ Constant TC

Summation

$(\text{sum}) \sum_{i=1}^{n} 1 \Rightarrow \underbrace{1+1+\ldots 1}_{n \text{ times}}$

$\Rightarrow n$

$= O(n)$



Topic : Analysis of Algorithms

$$f(n)=\prod_{i=1}^{n}(i)=1\times 2\times 3\times 4.....n$$

$$=\boxed{n!}$$

$$F(n)=O(n!)$$

$$=O(n^n)$$

$1\times 2\times 3\cdots n$

$n(n-1)(n-2)\cdots 1$

$=n!$

$\approx n^n$



Topic : Analysis of Algorithms

$n!$ vs n^n

$n \leq n \quad , n \geq 2$

$n*(n-1) * (n-2) \leq n * n * n$

$n * (n-1) \ldots 1 \leq n*n*n* \ldots\ldots\ldots\ldots *n$ (n times)

$n! \leq (n^n)$

$n! = O(n^n)$

n times n

n^n

$n! < n^n$

Is $n! = \Omega(n^n)$?

$n! \geq c * n^n$, $c > 0$

$n(n-1) \cdots * 1 \geq c * n^n$

$n(n-1) < n * n$

not true.

$n! \neq \Omega(n^n)$

$$n! = O(n^n)$$

But $n! \neq \Omega(n^n)$

Hence $n! \neq \Theta(n^n)$

MSQ

H.w

$A(n)$, $w(n)$

Always false?

A) $A(n) = o(w(n))$

B) $w(n) = O(A(n))$

C) $w(n) = \Omega(A(n))$

D) $w(n) = o(A(n))$

Ans: D

A) $A(n) = o(w(n)) \Rightarrow A(n) < C * w(n)$

B) $w(n) = O(A(n))$ $w = A$ ↳ may or may not follow

C) $w(n) = \Omega(A(n))$

$W \geq A$ → always True

D) $w(n) = O(A(n))$

$W < A \rightarrow A > W$ → always False

(Q) $\prod_{i=1}^{n} \log(i)$



$= \log(1) \times \log(2) \times \log(3) \cdots \log(n)$

$= 0 \Rightarrow \underline{O(1)}$

(8) $f(n) = \sum_{i=1}^{n} \log(i)$

$= \log(1) + \log(2) + \log(3) \cdots + \log(n)$

$= \log(1 \times 2 \times 3 \cdots \times n)$

$\log(a) + \log(b) = \log(ab)$

$= \log(n!) < \log(n^n) \Rightarrow \underline{(n * \log n)}$

is $\sum_{i=1}^{n} \log(i) = \Omega(n \log n)$?

True

Stirling's approximation:

$$n! \approx \sqrt{2\pi n} * \left(\frac{n}{e}\right)^n$$

$$\sum_{i=1}^{n} \log(i) = \log(n!)$$

$$= \log\left(\sqrt{2\pi n} * \left(\frac{n}{c}\right)^n\right)$$

$$= \log(\sqrt{2\pi n}) + \log\left(\left(\frac{n}{c}\right)^n\right)$$

$$= \log(\sqrt{2\pi}) + \log(\sqrt{n}) + n * \log(n/c)$$

$$= \text{const} + \frac{1}{2}\log(n) + n * [\log n - \log c]$$

$$= \left[\text{const} + \frac{1}{2}\log n + n\log n - n\log c\right]$$

$\Rightarrow O(n\log n)$

$\Rightarrow \Omega(n\log n)$

$\Rightarrow \Theta(n\log n)$

$n\log n$ → dominating term

$$\log(n!) = O(n \log n)$$
$$= \Omega(n \log n)$$
$$= \underline{\underline{\Theta(n \log n)}}$$

Imp practice: T/F

1) $2^{(n+1)} = O(2^n)$ T

2) $2^{(2n)} = O(2^n)$ F

3) $n^p = O(n^q)$ if $0 < p < q$ True

4) $2^{(n^2)} = O(n!)$ False

1) $2^{(n+1)} = O(2^n)$

$$\boxed{a^{(m+n)} = a^m * a^n}$$

$2^n * 2^1$

$= 2 * 2^n = O(2^n)$ ✓

2) $2^{(2n)} = O(2^n)$

$a^{m*n} = (a^m)^n$

$(2^2)^n$

$= 4^n > 2^n$

$4^n \neq O(2^n)$

$2^{2n} = 2^{(n+n)} = 2^n * 2^n$

$2^n \times 2^n > 2^n$

$2^n > 1$

$0 < p < q$ $\quad n^p = O(n^q)$

$p < q$

$n^p < n^q$

$n^p = O(n^q)$

4) $2^{(n^2)} = O(n!)$

$n! \approx n^n$

$2^{n^2} = O(n^n)$?

Take log

$\log_2(2^{n^2}) > \log_2(n^n)$

$n^2 \log_2 2^2 \quad n \log_2 n$

$n^2 \quad n \log n$

$n * n \quad n \log n$

$n > \log n$



Topic : Analysis of Algorithms

The big notation (O, Ω) provides the upper bounded & Lower Bound that may or may not be tight.

Tight LB:

(Can be as well as loose bounded)

E.g.1. $\boxed{\Omega(n)} \rightarrow \boxed{n+10} \rightarrow \boxed{O(n)} \rightarrow$ Tight UB

$\Omega(\sqrt{n})$ $O(n^2)$

$\Omega(1)$ $O(n^3)$

$\rightarrow$ Lose UB



Topic : Analysis of Algorithms

2. Small notations always provide bounds that are loose bounds.

↓

(never tight bound)



Topic : Analysis of Algorithms

1. Small – Oh Notation → Loose upper Bound

⇒ f(n) is O(g(n)) iff,

Small oh

for all $c > 0 \Rightarrow F(n) < c*g(n)$

Whenever $n \geq n_0$

for some $n_0 \geq 0$



Topic : Analysis of Algorithms

E.g.2. Diff. between O and o.

$n < c*n$

if $c = 1$, $n < 1*n$ False

Hence, $n \neq o(n)$

$n = o(n)$

$n = o(n^2)$

$n = O(n^2)$ → small oh

1. $f(n) = n^2 + n + 5$

$O(n^2) \rightarrow$ Tight Bound

$F(n) = O(n^3)$

$= O(n^2)$

$= O(n^2)$

$= O(n^3)$

$= O(n\sqrt{n})$

} small oh

$n^2+n+5 = O(n^2)$ ✓

$= O(n^3)$ ✓

$n^2+n+5 < c*n^2, \quad c>0$

$c=1$ → true

$= o(n^2)$ ✗

$= o(n^3)$ ✓

$n^2 \quad n\sqrt{n}$

$n > \sqrt{n}$

$= o(n\sqrt{n})$? ✗

$= O(n\sqrt{n})$ ✗

$n^2 < n^3$

Misconcept

$n^2 + n + 5 = o(n^3)$

$n^2 + n + 5 < c * n^3$

X

for all $c > 0$

$c = 1, n = 1$ X

$\rightarrow$ $1 + 1 + 5 < (1 \times 1)$

$7 < 1$ X



Topic : Analysis of Algorithms

2. Small Omega (ω) Notation → Loose lower Bound

$\Rightarrow$ f(n) is ω (g(n)) iff,

For all c > 0, f(n) > c* g(n), when never $n \geq n_0$

n_0 → ~~There sold~~

Threshold



Topic : Analysis of Algorithms

Eg.3. Big Omega(Ω) vs small Omega (ω)

$$f(n) = n \Rightarrow f(n) = \Omega(n) \quad \checkmark$$

$$= \Omega(\sqrt{n}) \quad \checkmark$$

$$= \Omega(n^2) \quad \times$$

$$= \omega(n) \quad \times$$

$$= \omega(\sqrt{n}) \quad \checkmark$$

$$= \omega(1) \quad \checkmark$$

$f(n) = n = \Omega(n)$

$n > c * n$



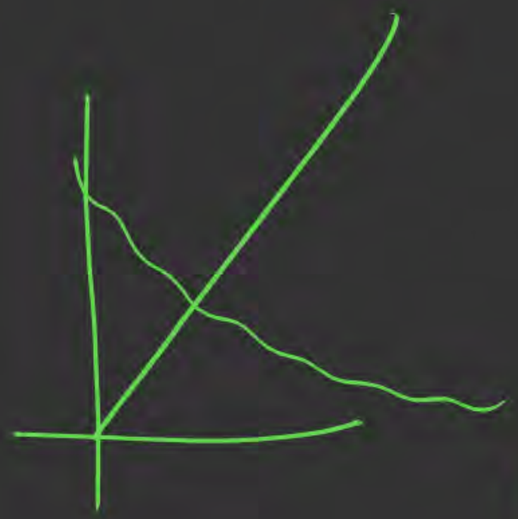
Topic : Analysis of Algorithms

Imp. Practice Questions

#Q. $2(2^n) = O(2^n)$

True

$2 \times 2^n = O(2^n)$ ✓



Is $n = \omega(1/n)$? True

$1/n$ tends to zero

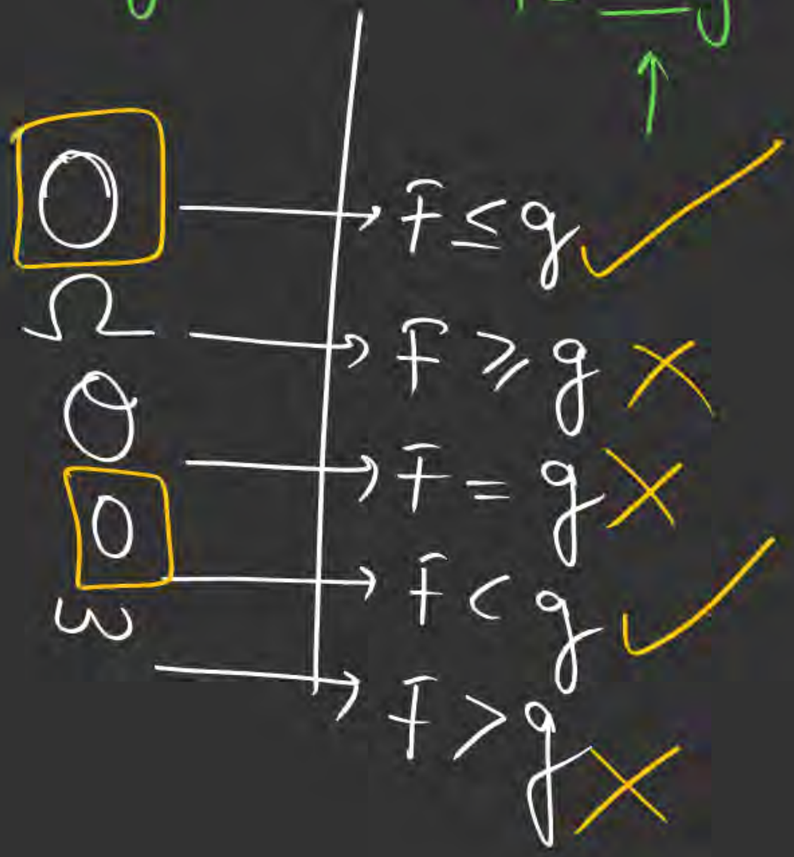
linear

$n > a\,1/n$

(8) $f = n * 2^n$

$g = 4^n$

$f = __g$



$f = n \times 2^n$ $\quad g = 4^n$

$n \times 2^n$ $\quad 4^n$

$(2^2)^n = 2^{2n} = 2^{n+n}$

$n \times \cancel{2^n}$ $\quad \cancel{2^n} * 2^n$

$n < 2^n \Rightarrow f < g$

$f < g$

$f \leq g$ X

less or =

$n \times 2^n$ $\quad$ 4^n

$\log_2(n \times 2^n)$ $\quad$ $\log_2(4^n)$

$\log_2 n + \log_2(2^n)$ $\quad$ $n * \log_2(4)$

$n + \log n$ $\quad$ $2n$

~~n~~ $+ \log n$ $\quad$ ~~n~~ $+ n$

$\log n < n$



1) every o is also O | but O may/may not be o.

2) every ω is also Ω | but Ω may/may not be ω.

$n^2 + n \to O(n^2) = o(n^2)$

Imp:-

$f = n^2 \log n$

$g = n^{500}$

$n^2 \log n \quad n^{500}$

$\cancel{n^2} \log n \quad \cancel{n^2}\, n^{498}$

$\log n < n^{498}$

$f < g$

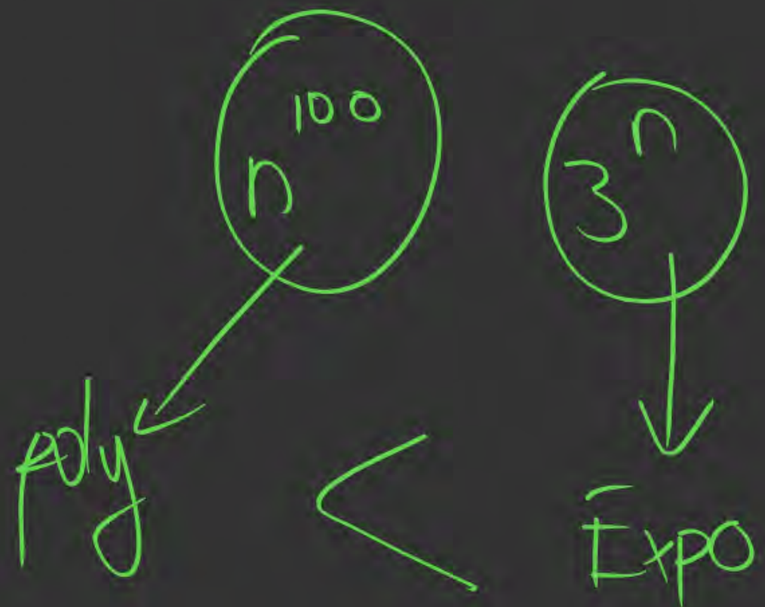
$f = O(g)$

$f = o(g)$

$f = n^{100}$

$g = 3^n$

n^{100} 3^n



$n^{100} < 3^n$

Take log

$\log(n^{100})$ $\log(3^n)$

$100 \log n < n \log 3$

$f < g$

$f = 2^n$

$g = n^{(\sqrt{n})}$

2^n $\quad$ $n^{\sqrt{n}}$

Take log

$\log_2(2^n)$ $\quad$ $\log(n^{\sqrt{n}})$

$n \times 1$ $\quad$ $\sqrt{n} \times \log n$

$\sqrt{n} * \sqrt{n}$ $\quad$ $\sqrt{n} > \log n$

$f > g$

$f = \Omega(g)$

$f = \omega(g)$

P48 → HW

$f_1 = 10$

$f_2 = \sqrt{n}$

$f_3 = \log n$

$f_4 = n$

$f_5 = 1/n$



incr of asymp growth



Summary

→ Asymptotic Notations

→ PYQs + Practice



THANK - YOU